Quick detour: built-in modules

Today

Welcome to your next lesson: Quick detour: built-in modules.

I'm preparing your personalized lesson based on what you've learned, what's ahead, and the best way to build your skills.

Step by step, you’re moving forward.

Imagine you have a few toolboxes on a shelf: one for math tools, one for randomness, one for time/date 🧰

In Python, each of those toolboxes is a module: math, random, time, datetime. Your program can say: “Hey, I want the math toolbox,” and then grab a specific tool from it, like sqrt.

Here’s the basic flow we’ll use:

Your Python file (your program)
You write import math → this pulls in the math toolbox
You use a tool: math.sqrt(16) → Python gives you the result
To warm up: does this idea of “modules as toolboxes of related code” still feel clear from before, or should I quickly re-explain it with a fresh tiny example?

You can just answer: “clear” or “please re-explain”.

Clear

Nice 👍 since the toolbox idea is clear, let’s use it with a new module.

Think about this tiny script:

Python
import random

```
secret = random.randint(1, 10)
print("Secret number is:", secret)
Step by step, what’s happening is:


import random → you grab the random toolbox.
random.randint(1, 10) → you use one specific tool from that toolbox to get a random number between 1 and 10.
print(...) → your own program shows the result.

So here, the module is random, and randint is one tool inside it.

Before we code anything:

Does it make sense that random (the module) is the toolbox, and randint is just one tool inside it that your program uses?

Yes, that makes sense

What do you think? Pick the option that best matches how we’ve been talking about modules as “toolboxes” 👇

Which statement best describes a Python module?


A.
One individual object created from a class


B.
A collection of related Python code


C.
A loop that repeats code


D.
A value stored in a variable

Nice, Ron — that’s exactly right 🎯


A Python module is a collection of related Python code:


```

It can contain functions, variables, and other things that all belong to the same “theme” (like math stuff in math, random stuff in random).
When you import a module, you’re adding that whole toolbox of related tools to your program.

The other options don’t fit because they describe:

One object from a class → that’s an instance, not a module.
A loop → that’s control flow.
A value in a variable → that’s just data, not a toolbox.
Want to do a tiny hands-on next, like using random or math in the code editor and walking through what the module is contributing? 😄

Let's try random

Love it, random is fun 😄🎲

Let’s do this step by step:

Open the code editor for this lesson.
In the editor, type this exact code:
Python
import random

secret = random.randint(1, 10)
print("Secret number is:", secret)
First click Run (just to see what happens).
Then click Submit code so I can see it and give feedback.
After you run it, the number will change each time.

Remember: random is the module/toolbox, and randint is one tool inside it.

Go ahead and do that in the editor, then submit the code.

Python
import random

secret = random.randint(1, 10)
print("Secret number is:", secret)
Nice work running that, Ron 🎲

Let’s label the pieces clearly:

random → the module (the whole randomness toolbox)
randint → one tool inside that toolbox
Your code (secret = ..., print(...)) → your program using the toolbox

So the module doesn’t print anything by itself; your code decides how to use its tools.

Quick check-in (no quiz, just your words):

In this little script, what does the random module add that your program alone couldn’t easily do?

the ability to generate a random number. i dont have to code an elaborite algo based on something like current time to do so

Exactly. Beautifully put 👌

Your program brings the idea (“I want a random secret number”), and the random module brings the skill (a tested way to generate randomness) — so you don’t have to build a messy algorithm yourself.

Let’s do one more small step with random to lock this in:

In the code editor, change your code to this:

Python
```
import random

secret = random.randint(1, 10)
guess = random.randint(1, 10)

print("Secret number is:", secret)
print("Computer's guess is:", guess)
```
Then:

Click Run to see what happens a couple of times.
Click Submit code so I can check it.

This will use the same module (random) to get two different random numbers.

Python
import random

secret = random.randint(1, 10)
guess = random.randint(1, 10)

print("Secret number is:", secret)
print("Computer's guess is:", guess)
Nice, Ron — that’s exactly how to use the module 💪

You just used the same module toolbox (random) to grab two tools calls:

random.randint(1, 10) for secret
random.randint(1, 10) again for guess

Even if they sometimes come out the same (like 1 and 1), they’re two separate random draws.

To close this mini-detour on modules, I want to check one last thing in your own words:

Very short answer:

In this script, what is the module, and what are the values?

(You can answer like: “Module: X, Values: Y and Z…”)

module is random.randint() vales are 1 and 10 sent in and secret and result, the end results

You’re very close here, Ron — just a tiny tweak 🛠️

Let’s separate module, function, and values:

Module: random
Function inside the module: randint
Together you call it as random.randint(1, 10)

Values (data) in your run:
Inputs to the function: 1 and 10
Outputs (example from your run):
secret might be 1
guess might be 1

So the module is not random.randint() — that's a function call from the module. The module is just random.

Want to try one more tiny practice example, this time with the math module toolbox (like math.sqrt), or do you want to stick with random for one more round?

One more random example

Nice, let's squeeze one more fun random example in 🎲

We'll make a tiny "dice roller" that rolls two dice and shows their total.

In the code editor, write this program:

Python
```
import random

die1 = random.randint(1, 6)
die2 = random.randint(1, 6)
total = die1 + die2

print("Die 1:", die1)
print("Die 2:", die2)
print("Total:", total)
```
Then:

Click Run a couple of times to see different results.
Click Submit code so I can see your version.

After you do that, I'll ask you one simple question about what part is the module and what part is your own logic.

Python
```
import random

die1 = random.randint(1, 6)
die2 = random.randint(1, 6)
```

```
total = die1 + die2

print("Die 1:", die1)
print("Die 2:", die2)
print("Total:", total)
```

Nice roll 😄 Even snake eyes still count as a correct program.

Let’s separate roles very clearly in this dice game:

Module toolbox: random → gives you the ability to generate random numbers.

Tools from the module: random.randint(1, 6) → used twice to simulate each die.

Your own logic: total = die1 + die2 and all the print(...) lines → that’s your program deciding what to do with the random numbers.

So: random supplies the numbers, you decide to add them and print them.

You’ve used modules like a pro here 🧰

Are you feeling ready to switch to a quick example with another module like math next, or would you rather do one last random twist (for example, a simple “You win if total ≥ 8” idea)?

Try the math example

Nice, let’s bring in the math toolbox 🧱

We’ll do something simple: use math.sqrt to get a square root.

Here’s the plan, step by step:

In the code editor, type this:

```
import math

number = 16
root = math.sqrt(number)
```

```
print("Number:", number)
print("Square root:", root)
Click Run to see the output.

Then click Submit code so I can check it.

After you run it, I'll ask you to tell me (very briefly) which part is the module and which part is just your program logic.

Python
import math

number = 16
root = math.sqrt(number)

print("Number:", number)
print("Square root:", root)
Perfect, Ron ✅


Let's break it down clearly:


Module: math → this is the toolbox you imported with import math.
Tool from the module: math.sqrt → a specific tool inside the math toolbox that calculates square roots.
Your program's own logic/data:
number = 16 → you choose the input
root = math.sqrt(number) → you use the tool
the two print(...) lines → you decide what to show

The module doesn't decide to print or pick 16 — that's all you.

To close this detour cleanly, I want to check your understanding with one last quick multiple-choice question about modules. What do you think? 👇

Sure, ask me

What do you think? Pick the best answer below 👇

Which statement best describes a Python module?


```

A.
One individual object created from a class

B.
A collection of related Python code

C.
A loop that repeats code

D.
A value stored in a variable

Nice job, Ron — you nailed it again 🎯

You chose “A collection of related Python code”, which is exactly what a module is:

It groups related tools (functions, variables, etc.) into one place, like random, math, time, datetime.
You bring that toolbox into your program with import module_name and then use its tools with module_name.tool(...) (like math.sqrt(16) or random.randint(1, 10)).

The other options describe things from other parts of Python (objects, loops, variables), not a toolbox of reusable code.

You’ve:

Used import correctly with random and math ✅
Shown you understand a module = toolbox of related code ✅

So this detour on built-in modules is complete 🎉

When you’re ready, you can jump back into the main Object-Oriented Programming track in the Learn tab and continue from there.

```
import random

secret = random.randint(1, 10)
guess = random.randint(1, 10)

print("Secret number is:", secret)
print("Computer's guess is:", guess)

import random

die1 = random.randint(1, 6)
die2 = random.randint(1, 6)
total = die1 + die2

print("Die 1:", die1)
print("Die 2:", die2)
print("Total:", total)


import math

number = 16
root = math.sqrt(number)

print("Number:", number)
print("Square root:", root)
```